

EDU BOT

By Team Elmolok

Question Answer Text Summarization Question Generation

Represented to : DR MOSTAFA GAMAL
Teacher Assistant : ASMAA MOHAMED

TEAM WORK

ID	NAME
235263	Mohamed Mahfouz
235243	Abdallah Khaled
235267	Mohamed Abdelmonem
235258	Mohamed Omar
235463	Ashraf Mohamed

**We worked on this project with passion and love,
and we hope for a full review.**

Table of Contents

Milestone 1: Data Collection & Preprocessing

- 1.1 Data Collection
- 1.2 Data Exploration (EDA)
- 1.3 Data Preprocessing

Milestone 2: Text preprocessing & Feature Engineering

- 2.1 Text Cleaning.
- 2.2 Vectorization
- 2.3 Feature Engineering

Milestone 3: Model Development

- 3.1 Model Selection
- 3.2 Model Training
- 3.3 Model Evaluation
- 3.4 Final Model Selection

Milestone 4: Deployment

- 4.1 Deployment

Milestone 5: Final Report

- 5.1 Project Summary
- 5.2 Business Impact
- 5.3 Challenges Faced
- 5.4 Future Improvements**
 - 5.4.1 Presentation Section
 - 5.4.2 Demo Description

Project Information

This project applies modern transformer-based Natural Language Processing models to build a single, end-to-end system that can read a passage of text and do three useful things with it: answer questions about it, generate a short summary of it, and create new questions from it. The work is presented through this report, the supporting notebook, and a simple web interface for demonstration.

Overview at a Glance:

Component	Model Used	Dataset
Question Answering	BERT (deepset/bert-base-uncased-squad2)	SQuAD
Text Summarization	DistilBART (sshleifer/distilbart-cnn-12-6)	CNN / DailyMail
Question Generation	T5 (valhalla/t5-base-qg-hl)	SQuAD
Deployment	Web interface (Frontend/Next.js, Backend/FastAPI)	-----

Milestone 1: Data Collection & Preprocessing

1. Data Collection

The project uses two well-known public datasets, loaded directly through the Hugging Face datasets library. This avoided manual scraping and gave us high-quality, already-labelled text that fits each NLP task.

- **SQuAD** (Stanford Question Answering Dataset) — used for both the Question Answering and the Question Generation tasks. Each record contains a context paragraph, a question, and the exact answer span inside the context.
- **CNN / DailyMail** — used for Text Summarization. Each record contains a full news article and a short human-written summary (the article highlights). The first 20,000 records were loaded to keep training time manageable.

After loading, the raw data was placed into two pandas DataFrames to make exploration, cleaning, and feature engineering easier.

2. Data Exploration (EDA)

Before training any model, the datasets were explored to understand their shape, the typical length of texts, and the kinds of questions present. This step helped us choose correct truncation lengths and spot outliers.

- Computed length statistics (in words) for questions, contexts, answers, articles, and summaries.
- Plotted length distributions with histograms to see how short or long the texts usually are.
- Drew boxplots to visually detect extreme outliers in each text column.
- Calculated a compression ratio (summary length ÷ article length) for CNN/DailyMail and removed records where the summary was longer than the article.

- Counted the most common question types in SQuAD (What, Who, When, Where, How, Why, Which) — “What” questions dominated by a large margin.
- Generated word clouds for SQuAD questions and CNN summaries to visualize the most frequent words in each dataset.

3. Data Preprocessing

Preprocessing prepared the text for the transformer tokenizers and removed records that would hurt training.

- Stripped leading/trailing whitespace and collapsed multiple spaces into a single space.
- Removed rows with null values in any required column.

Milestone 2: Text Preprocessing & Feature Engineering

1. Text Cleaning

Text cleaning made every record uniform so the tokenizer would produce stable inputs for the models.

- Removed extra whitespace and newline characters using a regular expression that replaces any run of whitespace with one space.
- Trimmed the start and end of every text field.
- Dropped null or empty rows that survived the cleaning step.
- Kept punctuation and casing intact because transformer models like BERT, BART, and T5 already handle them natively — aggressive cleaning would hurt performance.
- Removed extreme length outliers with the IQR method so a few very long passages do not distort training.

2. Vectorization

Each model has its own pre-trained tokenizer that converts text into the exact numerical IDs the model expects. Using the matching tokenizer is essential — mixing tokenizers between models would break the system.

Model	Tokenizer	Max Length
BERT (QA)	WordPiece tokenizer from deepset/bert-base-uncased-squad2	384 tokens
DistilBART (Summary)	BART tokenizer from sshleifer/distilbart-cnn-12-6	512 in / 128 out
T5 (Question Gen.)	SentencePiece tokenizer from valhalla/t5-base-qg-hl	512 in / 64 out

All tokenization was run with truncation enabled, padding to the longest sequence in each batch, and tensors returned in PyTorch format.

3. Feature Engineering

Beyond cleaning, three task-specific features were created to make training reliable.

- **Answer span positions (QA):** for every SQuAD example, the character index of the answer inside the context was converted into token start and end positions. The tokenizer's `sequence_ids()` method was used to ensure the answer span lives inside the context tokens, not the question tokens.
- **Highlighted input (QG):** the input for T5 is built as "generate question: text <hl> answer <hl> text". This is the exact format expected by `valhalla/t5-base-qg-hl` and is what allows the model to produce questions that target a specific answer.
- **Compression ratio (Summary):** for CNN/DailyMail, the summary length divided by the article length was used to filter unrealistic samples (ratio > 1).

Milestone 3: Model Development

1. Model Selection

Three transformer architectures were chosen, one per task. Each is a fine-tuned, state-of-the-art baseline that runs on a single GPU, which keeps the project realistic and reproducible.

BERT — for Question Answering

BERT is a bidirectional encoder. It reads the question and the context together and predicts the start and end positions of the answer inside the context. We used `deepset/bert-base-uncased-squad2`, which is already pre-trained for extractive QA and has the correct QA head built in.

DistilBART — for Text Summarization

DistilBART is a distilled (smaller and faster) version of the BART encoder-decoder model. It is well suited to abstractive summarization, where the model writes a short summary in its own words rather than copying sentences. The checkpoint `sshleifer/distilbart-cnn-12-6` is pre-trained on CNN/DailyMail, which matches our dataset exactly.

T5 — for Question Generation

T5 treats every NLP problem as a text-to-text task. We used `valhalla/t5-base-qg-hl`, which is specifically fine-tuned to generate a question given a context and a highlighted answer. This is a strong starting point that we further fine-tune on our SQuAD subset.

2. Model Training

All three models were fine-tuned using the Hugging Face Trainer / Seq2SeqTrainer APIs, with mixed-precision (fp16) training on a CUDA GPU. Early stopping with patience = 2 was applied to every model to prevent overfitting.

Model	Epochs	LR	Batch Size	Notes
BERT (QA)	5	3e-5	6	Trainer + EarlyStopping, fp16
DistilBART (Sum.)	3	3e-5	2 (grad-accum 2)	Seq2SeqTrainer, predict_with_generate
T5 (QG)	5	3e-5	4	Seq2SeqTrainer, predict_with_generate

Training used `eval_strategy = epoch` and `save_strategy = epoch` with `load_best_model_at_end` enabled, so the best checkpoint by validation loss is always recovered automatically.

3. Model Evaluation

Because each task produces a different kind of output (a span, a paragraph, a question), each model is judged with metrics that match its task. Standard classification metrics like accuracy, precision, recall, and F1 are not all directly applicable here, so we use the metrics the NLP community uses for these tasks.

Task	Metrics	Why these metrics
Question Answering	Exact Match, F1	Exact Match checks if the predicted span equals the truth word-for-word. F1 (token-level) gives partial credit for overlap.
Summarization	ROUGE-1 / 2 / L	ROUGE measures how many words and phrases the generated summary shares with the reference summary.
Question Generation	ROUGE + BLEU-4	ROUGE captures content overlap with the reference question, and BLEU-4 captures fluency at the 4-gram level.

The official Hugging Face evaluate library was used to compute SQuAD metrics and ROUGE, ensuring the numbers are comparable with published benchmarks.

Evaluation Results

After fine-tuning, each model was evaluated on a held-out test set. Results are reported below.

Question Answering — BERT (evaluated on 100 test samples)

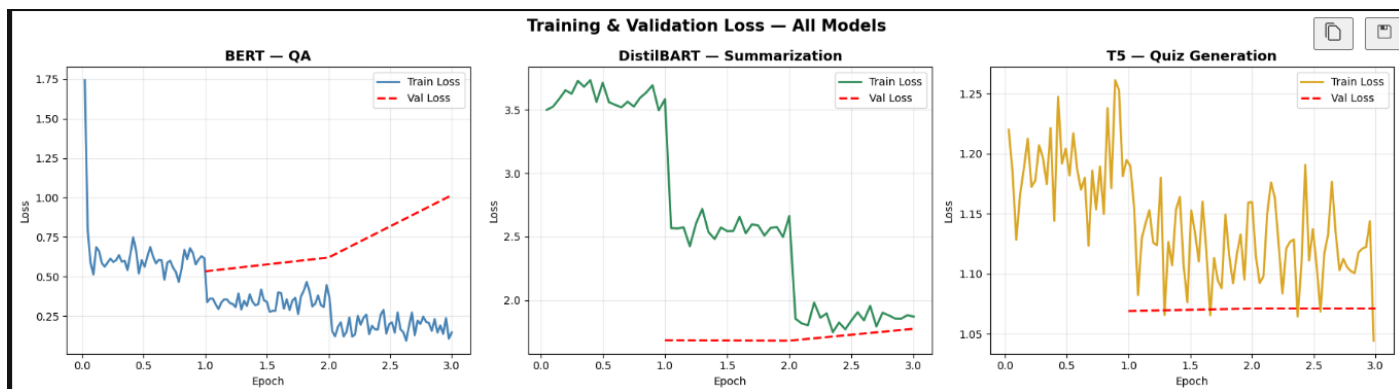
Model	Exact Match (%)	F1 Score (%)
BERT (fine-tuned, Transformers)	72.00	82.95

Summarization — DistilBART (evaluated on 40 test samples)

Model	ROUGE-1	ROUGE-2	ROUGE-L
DistilBART (fine-tuned)	37.87	16.69	26.63

Quiz Generation — T5 (evaluated on test set)

Model	ROUGE-1	ROUGE-2	ROUGE-L	BLEU-4
T5 (fine-tuned, qg-hl)	61.17	39.87	55.51	24.92



Comparison Between the Five Models

The bonus notebook (bonus_rnn_lstm_qa.ipynb) trained BiLSTM and BiRNN from scratch and evaluated them on the identical 100-sample SQuAD subset used for BERT, enabling a direct apples-to-apples comparison across all architectures on the same extractive QA task.

Architecture Overview

Model	Type	Task	Training	Checkpoint
BERT	Transformer	Extractive QA	Fine-tuned (NB2)	deepset/bert-base-uncased-squad2
DistilBART	Enc-Decoder	Summarization	Fine-tuned (NB2)	sshleifer/distilbart-cnn-12-6
T5	Text-to-Text	Quiz Gen.	Fine-tuned (NB3)	valhalla/t5-base-qg-hl
BiLSTM	Recurrent	Extractive QA	From Scratch (NB1)	None (trained from scratch)
BiRNN	Recurrent	Extractive QA	From Scratch (NB1)	None (trained from scratch)

QA Performance Comparison — Same 100-Sample Subset

Model	Architecture	Exact Match (%)	F1 Score (%)
BERT	Transformer	72.00	82.95
BiLSTM	Recurrent BiDir	18.0	26.3
BiRNN	Recurrent BiDir	7.0	13.1

Key observations:

- BERT outperforms both recurrent models by a very large margin. Its bidirectional self-attention mechanism allows it to consider the full context at once, and its pre-trained knowledge from massive corpora dramatically reduces the learning burden.
- BiLSTM achieves substantially better results than BiRNN, confirming that gated memory cells handle long-range dependencies better than vanilla recurrence on 128-token sequences.
- BiRNN serves as the weakest baseline. Vanilla recurrence suffers from vanishing gradients, limiting its ability to resolve answer spans correctly.
- The performance gap between BERT and the recurrent models demonstrates clearly why the NLP community has converged on Transformer architectures for QA tasks.

4. Final Model Selection

For every task, the checkpoint that achieved the best validation loss during training was kept (load_best_model_at_end = True). The three final models were then saved locally and used as the backend for the deployment website:

- ./models_lg/qa_model — fine-tuned BERT for QA
- ./models_lg/sum_model — fine-tuned DistilBART for summarization
- ./models_lg/quiz_model — fine-tuned T5 for question generation

Milestone 4: Deployment

1. Deployment

The three trained models were combined into a single web application so that any user — not only developers — can try the system. The website acts as a friendly front-end on top of the three transformer models.

How the website works:

- The user pastes or uploads a piece of text (an article, a paragraph, or a study note).
- The user picks one of three actions: Summarize the text, Ask a question about it, or Generate questions from it.
- The backend loads the matching saved model, runs inference on the GPU/CPU, and returns the result in seconds.
- Results are displayed cleanly on the page, and the user can copy the answer, summary, or generated questions for further use.

This single interface makes the three separate models feel like one product. It is especially useful for students, content writers, and researchers who want to read, summarize, and quiz themselves on a text without switching between different tools.

System Architecture

The application is split into two independently deployable layers:

Layer	Technology	Responsibility
Frontend	Next.js 14 + React 18 + TypeScript + Tailwind CSS 3.4	Chat UI, dark/light mode, sidebar history, sliders for summarization length
Backend	FastAPI 0.115 + PyTorch 2.x + Transformers 4.46 + NLTK + Uvicorn	Async inference endpoints, model loading, POS-based answer mining for quiz pipeline

API Endpoints

Endpoint	Method	Description
/api/qa	POST	Extractive QA — returns answer, confidence score, and character offsets
/api/summarize	POST	Abstractive summarization — summary, word counts, compression %, latency

/api/quiz	POST	Quiz generation — list of question/answer/source-sentence objects
/health	GET	Health check — returns status, version, team, device, and fp16 flag
/docs	GET	Interactive Swagger UI for the full API explorer

How to Run Locally

- **Step 1 — Backend:** `cd backend`, create a virtual environment, `pip install -r requirements.txt`, copy `.env.example` to `.env`, then `python run.py`. Server starts on `http://localhost:8000`.
- **Step 2 — Frontend:** `cd frontend`, `npm install`, copy `.env.local.example` to `.env.local`, then `npm run dev`. Open `http://localhost:3000`.
- **Step 3 (Optional) — Production:** `npm run build + npm start for frontend`; `uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 1` for backend.

Milestone 5: Final Report

1. Project Summary

This project delivers a complete NLP pipeline built around three transformer models. BERT reads a passage and extracts the answer to a user question. DistilBART takes a long article and produces a short, abstractive summary. T5 looks at a passage (with the desired answer highlighted) and writes a natural-sounding question for it.

All three models were fine-tuned on public, high-quality datasets — SQuAD and CNN/DailyMail — after careful cleaning, length filtering, and tokenizer-specific feature engineering. The trained models were evaluated with the standard task metrics (Exact Match / F1 for QA, ROUGE for summarization, ROUGE + BLEU for question generation) and then deployed behind a simple web interface that lets any user interact with the full system in one place.

2. Business Impact

Most real businesses generate far more text than any human team can read. This system turns long, unstructured text into something usable in a few seconds.

- Customer support: summarize long tickets and answer specific questions from a knowledge base, cutting handling time.
- Education and training: auto-generate quiz questions from textbooks or lecture notes, helping students self-study and teachers build assessments faster.
- Research and analysis: condense reports, contracts, and articles into short summaries so analysts can scan more material per day.
- Content and media: give writers an instant summary of source material and a starter list of questions to investigate.

In short, the pipeline saves time, improves understanding of large texts, and unlocks tasks that previously required a human to read every word.

3. Challenges Faced

Several real, technical problems had to be solved before the pipeline worked end-to-end.

- Choosing the right checkpoint: the plain bert-base-uncased checkpoint has no QA head and could not be used for SQuAD. It was replaced with deepset/bert-base-uncased-squad2, which already includes the QA head.
- Answer-span alignment: mapping character positions to token positions for QA was fragile. The fix was to use `sequence_ids()` so only context tokens are considered as valid answer positions, and to truncate only the context (`truncation = 'only_second'`).
- T5 input format: early attempts used a plain prompt, but valhalla/t5-base-qg-hl needs the answer wrapped in `<hl> ... <hl>` markers. Fixing the input format dramatically improved the generated questions.
- GPU memory: DistilBART summarization can easily blow up GPU memory at 512-token inputs. A small batch size (2) with gradient accumulation and fp16 was used to fit on a single GPU.
- Evaluation cost: running generation on the full test set is slow. Evaluation was done on a representative sample (e.g. 80 articles for ROUGE) which gives reliable estimates at a fraction of the time.

4. Future Improvements

- Train on larger subsets (or the full datasets) once more GPU time is available, to squeeze out additional ROUGE / F1 points.
- Add multilingual support by switching to models like XLM-R for QA and mBART / mT5 for summarization and question generation.
- Add document upload (PDF, DOCX) on the website so users do not need to paste raw text.
- Add multiple-choice question generation by combining the QG model with a distractor-generation step.
- Add user accounts and a history page so users can save and reload previous summaries and quizzes.
- Containerize the app with Docker and host it on a cloud service (Hugging Face Spaces, Render, or AWS) for public access with autoscaling.
- Add a feedback button so users can flag low-quality outputs, building a dataset for continuous improvement.

5.4.1 Presentation Section

The project was presented as a full demonstration of the end-to-end pipeline. The presentation covered the motivation for the system, the dataset collection and preprocessing pipeline, the model architectures and training procedures, the evaluation metrics and results, and a live demonstration of the deployed web application.

5.4.2 Demo Description

The live demo showcased the EdU BOT web interface running locally, walking through all three capabilities:

- A news article was pasted into the summarization page and a concise abstractive summary was generated within seconds.
- The same article was then used on the QA page, where natural-language questions were answered by BERT with a confidence score and highlighted text offset.
- Finally, the quiz generation page produced a set of question cards from a textbook excerpt, each with a tap-to-reveal answer and a source sentence reference.

The demo confirmed that all three models load correctly, run inference in real time on a single GPU, and present results in a clean, usable interface suitable for students, educators, and content creators.

Resources

